

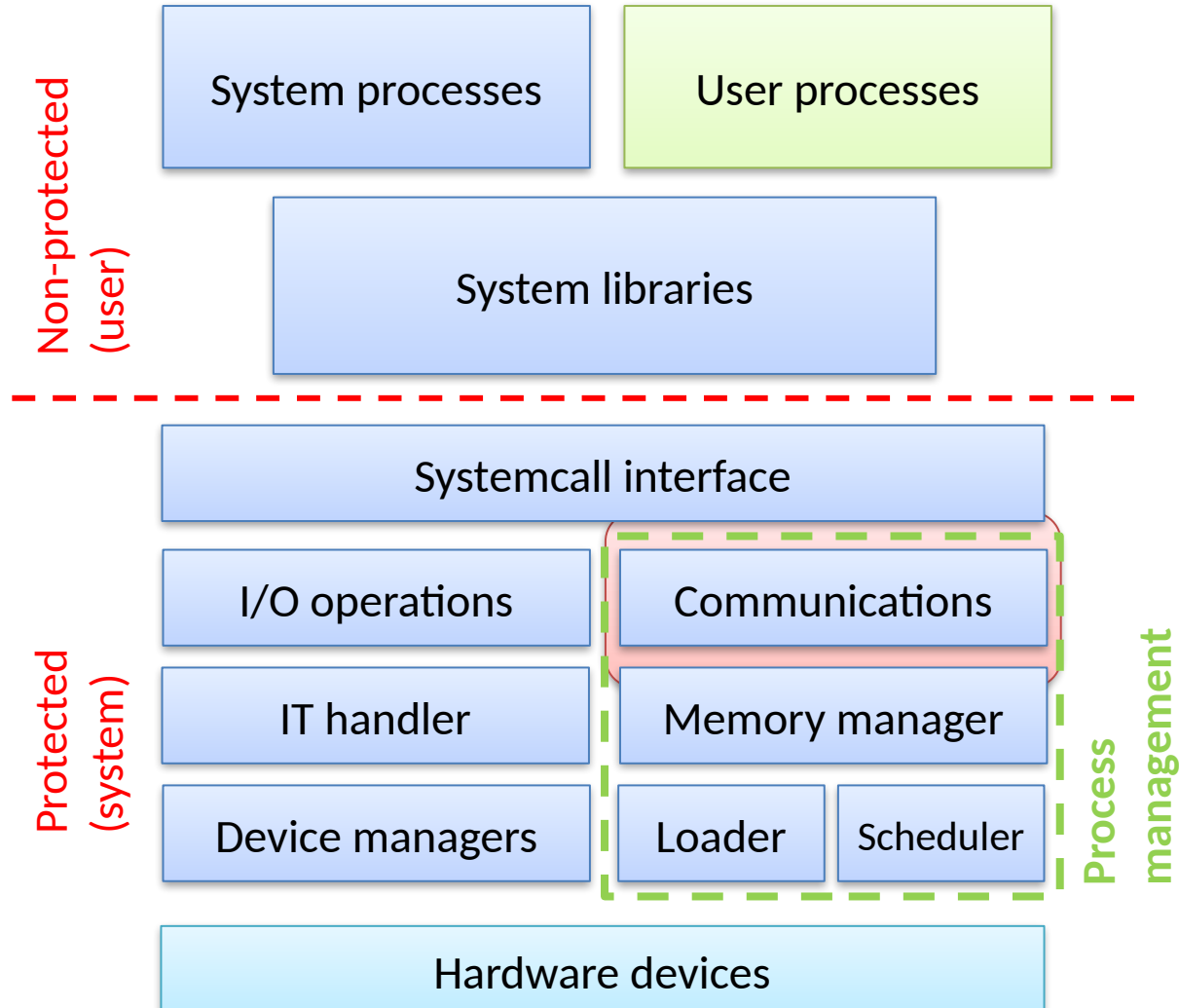
Operating Systems – Interprocess Communication

Lecturer: András Millinghoffer
milli@mit.bme.hu

Budapest University of Technology and Economics (BME)
Department of Measurement and Information Systems (MIT)

The slides of the latest lecture will be on the course page. (<https://www.mit.bme.hu/eng/oktatas/targyak/vimiab00>)
These slides are under copyright.

The main blocks of the OS and the kernel (recap)



The abstract virtual machine concept

- The main goal of the OS to support the execution of user jobs
 - Jobs are implemented by one or multiple tasks
 - Decomposed jobs may require communication between tasks
- Implementing tasks: processes and threads
 - Threads in the same process have a shared memory
 - Simple communication, but conflicts may arise
 - Processes are separated from each other
 - More complex communication
- Memory management separates the processes
 - Every process has its own memory range, which others cannot access
 - For efficient operation there can be overlapped ranges
 - Read-only pages can be used by multiple processes
 - The copy-on-write method accelerates the process creation
 - The OS contains mechanisms to support the cooperation of the tasks
 - Usually these are in the kernel, not available for application developers
 - Communication interfaces has to be provided for developers!

The basic forms of communication

- Through shared memory
 - The tasks memory range has a special range, which is translated to a shared physical memory
 - This shared range can be read and written by multiple tasks simultaneously
 - This method called: **PRAM model** (Pipelined RAM)
 - It can be used by multiple tasks
 - The parallel operations will be ordered (pipeline), no overlapping between them
 - Competitive situations (e.g. multiple writers at the same time)
 - The execution order of these operations are not defined
- With messaging
 - No shared memory
 - The OS provides the communication mechanisms
 - Many solutions for this problem
 - Operations
 - `Send(recipient, data_pointer[, data_size]);`
 - `Receive(sender, data_pointer[, data_size]);`
 - Examples
 - Network communication, distributed systems, microkernel

The PRAM (pipelined RAM) model

- Parallel tasks can use the shared memory range
 - They can start operations independently from each other
 - The operations may start at the same time
- Rules for the operations arrived at the same time
 - *Read-read*
 - Two simultaneous reads operations will both return the RAM contents
 - *Read-write*
 - The RAM will be written, the read op. will return the new value or the old value
 - *Write-write*
 - One of the two values will be written to the RAM, a third value cannot be created
- The effect of the rules
 - The operations have no effect on each other (no mix-up), hence the **pipelined** term: the operations are ordered in a pipeline
 - It's undetermined which op. will execute first if they arrive at the same time
 - To avoid undefined operations the tasks have to **synchronize** their operation

Communication methods based on the PRAM model

- Data exchange between the threads of the same process through global variables
 - Not supervised by the OS, the programmers design the operation
- Shared memory between processes (SHM)
 - The virtual memory range of the processes has a special part, which is translated to the same physical address
 - After the assignment, the usual memory operations can be used (no system calls, no additional overhead)
 - Fast method to communicate
 - The capacity is limited (???) (the size is defined by the kernel)
 - Standard: POSIX Shared Memory
 - Implementations: System V UNIX, Windows (part of the kernel)
 - Typical applications: database engines
- Memory mapped file (mmap)
 - Accessing a file through the memory: a shared memory which are translated to the same disk blocks
 - The virtual memory manager of the OS performs the assignment
 - It may used for performance acceleration also

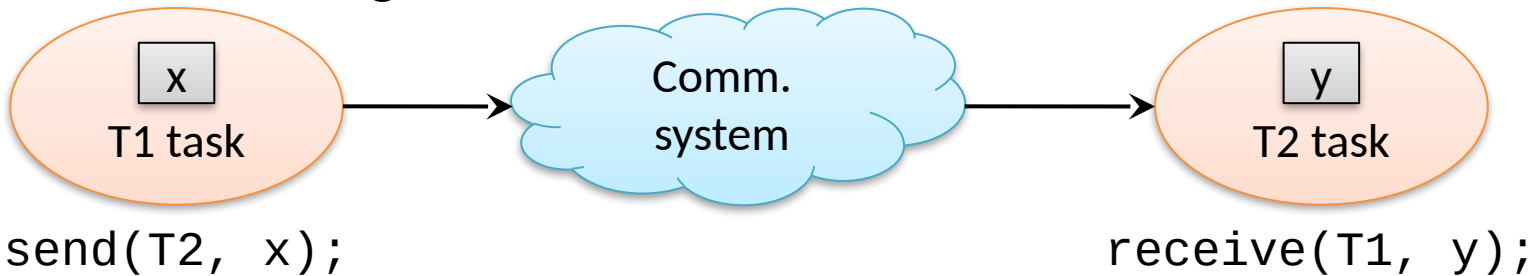
Communication with messaging

Basic questions of messaging

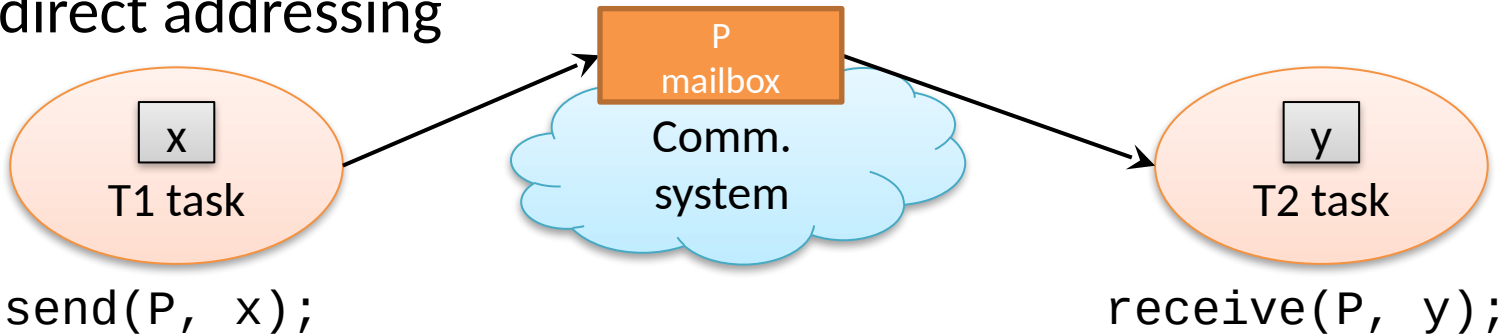
- Addressing: how can be the recipient address determined?
 - Direct approach or using a mediator?
 - Single addressing or broadcasting?
 - Simultaneous receiving from multiple senders?
 - If the recipient knows the sender, can it decide whether to receive the message or not?
- Synchronization
 - Is the `send()` function blocking or non-blocking?
 - If not, how can the sender know that the message is received?
 - How the `receive()` function works?
 - What happens if the `receive()` function is called, but the data is not yet arrived?
 - Are acknowledgements mandatory, when the data is received?
- Semantics of data exchange
 - The data stored by the sender, or the receiver, or both?
- Performance, reliability
 - What throughput can be achieved? How much is the delay between send and receive?
 - How many and what size messages can be sent?
 - Who supervise (error handling) the process? What happens if an error occurs?

The basic methods of addressing

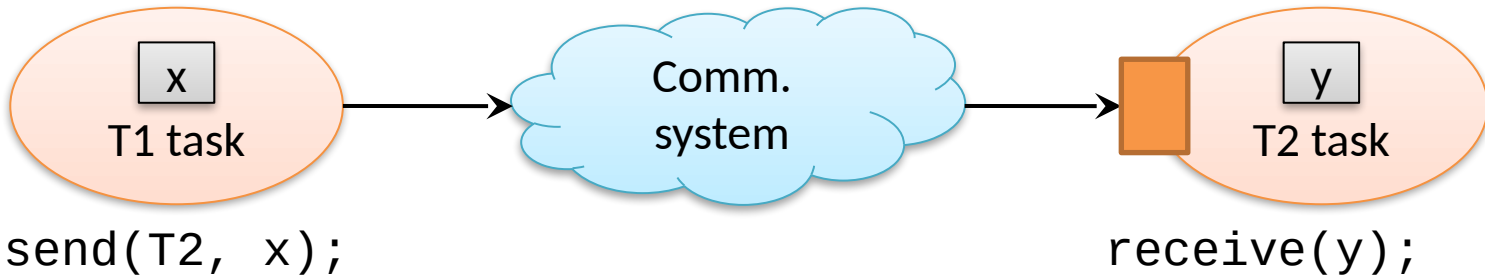
- Direct addressing



- Indirect addressing



- Asymmetric (direct on the sender's side) addressing



- Multiple addressing (multicast, broadcast)

Synchrony

- **Synchronous data transfer**

- The operation blocks the task execution (waiting state)
 - Send: it returns only, when the data is received
 - If there are no mediator too long blocking is possible
 - Receive: it returns only, when the data is received
- The receive operation is sensitive to the Denial-of-Service (DoS) attacks
- Timeouts and other errors has to be managed

- **Asynchronous data transfer**

- The tasks are not blocked during the transfer
- The results are not available when the function returns
- The tasks has to be notified by other methods about the completion or the errors
- The messages which are not received has to be stored
- Asynchronous operations
 - Send: returns immediately after calling
 - The successful delivery has to be checked later
 - Receive: returns immediately with the data or with „no data received”
 - Busy waiting for a message should be avoided

Semantics of data transfer

- Data is transferred between tasks (and between the comm. system)
 - How long the data is available for the participants?
- Possible solutions
 - Copy semantics
 - The sender and the recipient also stores a copy of the data
 - Further modifications of these data has only local effects
 - Share semantics
 - After the transfer the sender and the recipient can access to the same data
 - Beside the data, permissions can be transferred (e.g. who can modify?)
 - Usually the kernel don't copy the data, only pointers are transferred
 - Move semantics
 - The sender loses access to the data after the sending
 - It can be implemented with shared data and with revoking permissions

Direct and asymmetric communications

- Signal
 - The sender can signal an event to the recipient
 - The sender can be the kernel: sending signals to the tasks (e.g. kill)
- Socket communication
 - Asymmetric method based on the client-server model
 - It is widely used on TCP/IP bases
 - The machines identified by IP addresses, the senders and receivers identified by ports
 - The OS provides the network connectors for the communication
- Remote Procedure Call (RPC)
 - Asymmetric method based on the client-server model
 - Calling a function in another task
 - The sender transfers the function name and arguments and receives the return value
 - The recipient performs the operations with the provided arguments and returns the results
 - It is based on network communications, so it can be used between different machines
 - Besides the communication protocol, the data semantics also determined
 - Exact data types, structures
 - Automatic conversion may be performed between the participants
 - Higher level OS services based on RPC
 - A number of development schemes (design patterns) are based on RPC

Indirect communication solutions

- Mailbox
 - It can contain a limited number (may be just one) of messages
 - The size of a message is determined by the system
 - The mailbox has a direct address (which is not recipient address)
- Message queue
 - It can store and transfer infinite (of course is limited in the reality) number of messages
 - It may operate between different machines also, via network
 - It can also filter the messages
 - The message queue has a direct address (which is not recipient address)
 - There are high level business implementations also ([Message Oriented Middleware](#))
 - E.g.: MSMQ, IBM MQ, Java Message Service, Oracle Advanced Queuing, ...
- Pipe
 - It can transfer infinite amount of data
 - Continuous operation (no message separation)
 - The pipe has a direct address (which is not recipient address)
 - No filtering
 - The receiver may request a copy: the data stays in the pipe, otherwise it will be removed

Performance compared to the PRAM model

- Slower data rate and higher delay
 - The communication subsystem has an overhead
 - Less efficient than PRAM operations
- What determines the data rate and delays?
 - Data transfer
 - Moving data between the task's and the kernel's memory range is slower
 - Context changes
 - Repetitive communication will cause a high number of context changes
 - System calls
 - The messaging is performed by system calls, which has an additional overhead

Due to the protective mechanisms, the efficiency is lower

- Where is the performance of the messaging is critical?
 - Microkernels (it uses messaging for internal operations)
 - First implementations are based on RPC: even slower

Solutions in modern microkernels

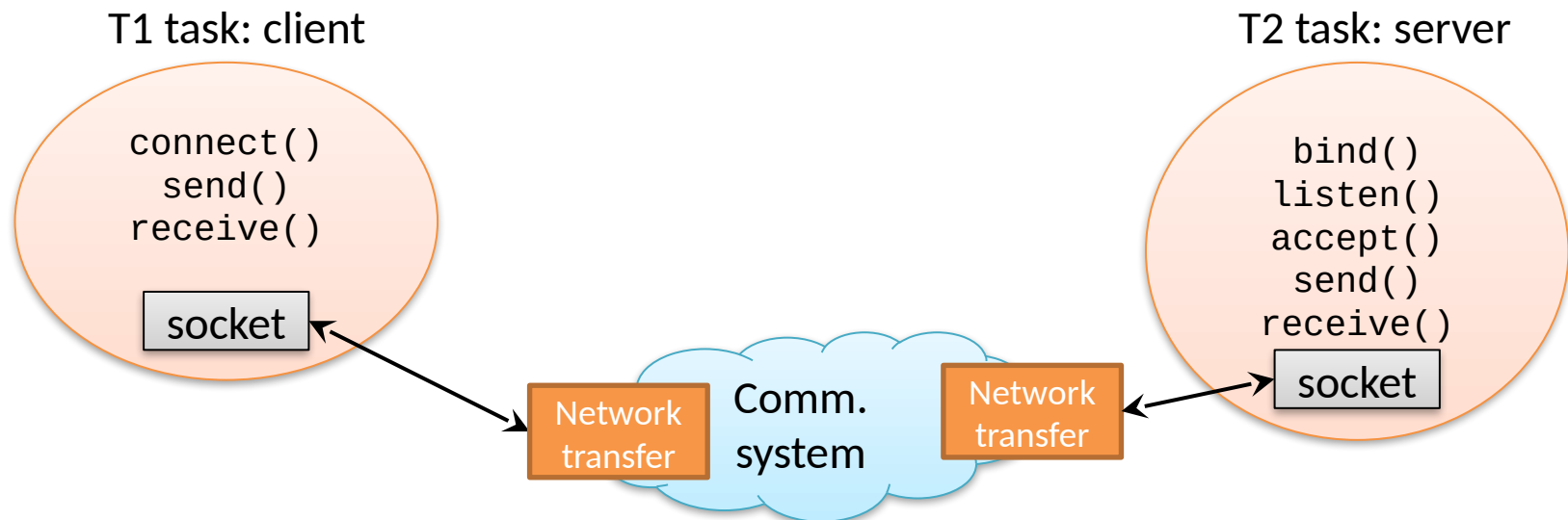
- What is the fastest way to transfer a small amount of data?
 - Registers of the CPU
 - Limited resource < 16 byte
- Transferring medium size data (16-64 byte)
 - Depending of the HW, it may fit in registers
 - L4 cache creates **virtual registers**
 - It can be translated to physical registers
 - Or it can be assigned to a special shared range in the tasks virtual memory range (PRAM)
- Lowering the overhead of the context changes (and scheduling)
 - The sender enters into waiting state, then: **direct context change**
 - Not the scheduler decides which task to run next
 - The kernel schedules the receiver task directly
 - The scheduling algorithms not have to run in this case
 - **Lazy queuing**
 - Direct context changes are performed during the communication cycles between the sender and the recipient
 - In the meantime the scheduler is not operating
 - It can go on until the communication finished or interrupted

The basic forms of communication (summary)

- **PRAM model**
 - Shared memory
 - The operations are pipelined
- Synchronous transfer operations
- No addressing
- Applications:
 - Threads between a process
 - **Shared memory between processes**
- Pro-s
 - Fast, simple
 - No overhead after initialization
- Con-s
 - R-W and W-W conflicts
 - Synchronization between participants is necessary
 - Limited capacity (??)
- **Messaging systems**
 - It uses a communication system
 - Send and receive operations
 - Parallel operations are ordered
- Synchronous or asynchronous operation
- Addressing: direct, indirect, multi
- Applications
 - Mailbox
 - Pipeline
 - Message queue
 - **Network socket**
 - **Remote Procedure Call**
- Pro-s
 - Widely available
 - Between different machines
- Con-s
 - Communication errors has to managed
 - Slower

Network socket communications

- The communication is based on network protocols and addressing
 - Between any process – client-server architecture
 - Between local processes: localhost, 127.0.0.1 or ::1 network address
 - Between remote computers
 - Multiple addressing (normal and multicast) and protocols (IP, TCP, UDP)
- The descriptor of the communication channel: **network socket**
 - The identifier of the communication endpoint in tasks



Remote Procedure Call (RPC)

- Distributed system infrastructure based on network communications
 - High level communication between processes
 - Remote function calls in different processes (even on a different machine)
 - Simple implementation due to common API-s
- Structures
 - Communication infrastructures
 - Protocols for transferring functions calls
 - Addressing for identification of the processes and functions
 - Format of the data (standard, independent of the HW, the OS and the implementation)
 - Portmapper: assigning process ID-s and ports
 - RPC language: the description of the procedures and data types
 - Procedure names, argument list (types) and return types
 - rpcgen: generates code from the interface specifications
- Example: [ONC \(former SUN\) RPC](#)

RPC interface specification and code generation

- RPC language (date.x file)

```

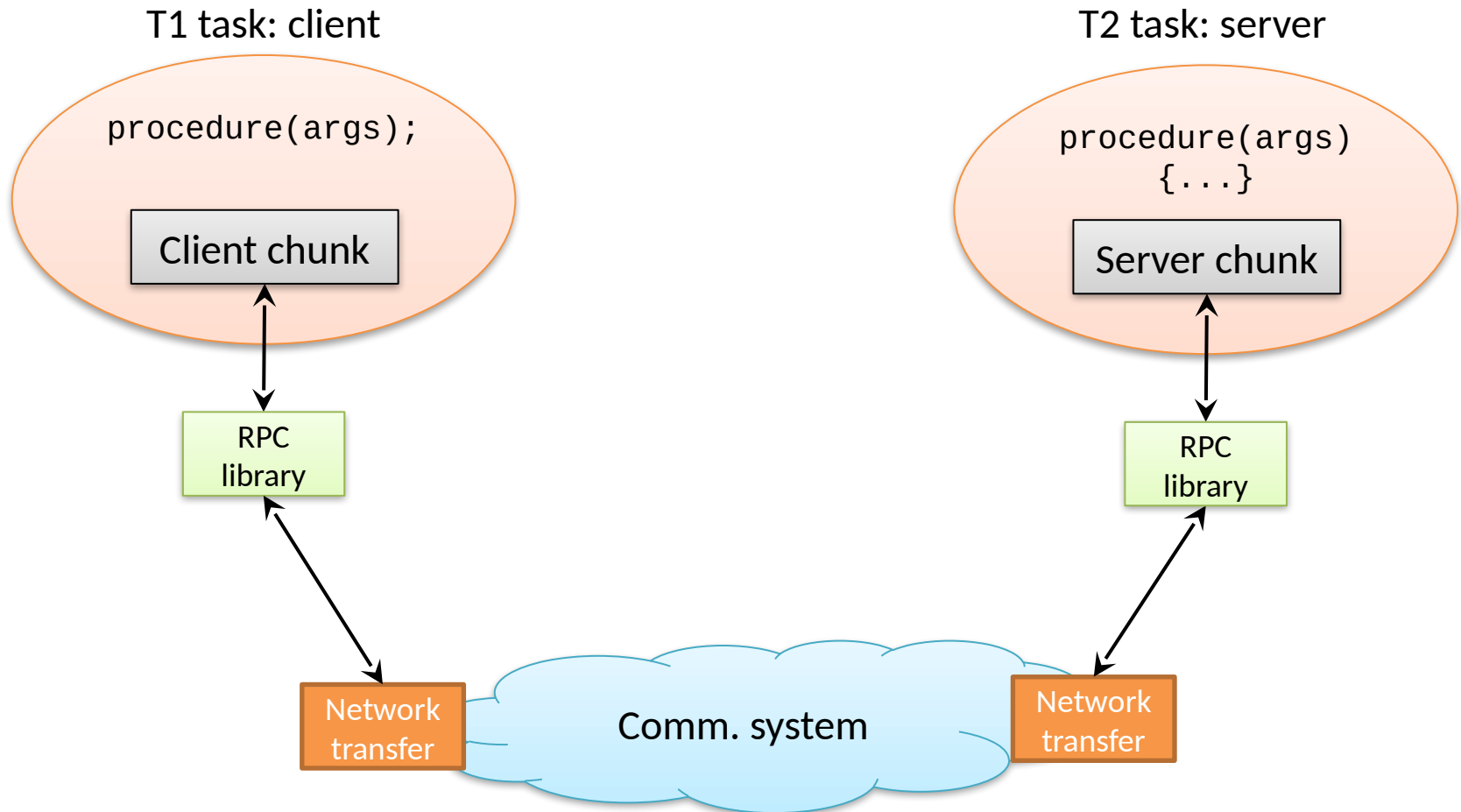
program DATE_PROG {
    version DATE_VERS {
        long BIN_DATE(void) = 1;    /* procedure ID = 1 */
        string STR_DATE(long) = 2; /* procedure ID = 2 */
    } = 1;                          /* version = 1 */
} = 0x31234567;                    /* process ID = 0x31234567 */
  
```

- Code generator: rpcgen

- Results of rpcgen date.x:

- date.h: declaration of the data types
- date_clnt.c: The declaration of the date_ functions in the client's code
- date_srv.c: the functions which will call the implementation of the date_ function in the server
- date_xdr.c: data manager procedures for the multi architecture XDR format

The blocks of the RPC communication



Solutions based on and inspired by RPC

- High level OS services
 - Network File System (NFS): see later lecture about file systems
- Inspired some application development design patterns
 - E.g.: CORBA, DCOM, Java RMI, .Net Remoting, D-bus, XML-RPC, SOAP, ...
 - Interface Description/Definition Language (IDL)
 - Independent description of the interfaces from programming languages
 - Widely used in distributed or component based systems
 - OMG IDL, WSDL, Microsoft MIDL, Facebook/Apache Thrift, ...
 - Also used in standards (platform independent)
 - W3C DOM IDL
 - Program code can be generated from the interface description
 - Available for almost all programming language on all architectures and OS-s

Interprocess communication in practice

UNIX signals

- Recipient: a process or a process group (see task management)
 - Notification about events in kernel, other processes, or in itself (e.g. errors)
- Type of the signals (SIGINT, SIGCHLD, SIGKILL, ...see: `kill -l`)
 - System: exceptions, quota, warning, notification
 - User: `ctrl+c`, `ctrl+z`
- Overview of the operation
 - Generating a signal (initiated by an interrupt, system call)
 - The kernel notifies the recipients about the signal
 - The recipient handles the signal in a procedure: it may ignore it, terminates, starts further processing, etc.
- Problems
 - Generation and delivery of the signal is delayed

Generating and handling signals

- Generating signals (by a process)

```
#include <signal.h>          /* kill() */
kill(pid, SIGUSR1);          /* sending signal */
```

- Handling signals

– Multiple handlers are possible

- Core: core dump and terminate (exit())
- Term: terminate (exit())
- Ign: ignore
- Stop: suspend
- Cont: continue from suspend

– The application may register custom handlers

```
signal(SIGALRM, alarm); //register the handler
void alarm(int signum){...} //the handler function
```

– The possibilities are limited by the type of the signal

- E.g. SIGKILL cannot ignored, and custom handler cannot registered

UNIX signals: examples

```
#include <signal.h>          /* signal(), kill() */
#include <unistd.h>           /* getpid() */
#include <sys/types.h>        /* pid_t */
pid_t pid = getpid();        /* own PID */

kill(pid, SIGSTOP);          /* sending STOP signal */
(Equivalent terminal command: kill -STOP <PID>)

signal(SIGCLD, SIG_IGN);     /* children ignored */

signal(SIGINT, SIG_IGN);     /* ctrl+c signal ignored */

signal(SIGINT, SIG_DFL);     /* registering default handler */

signal(SIGALRM, myalarm);    /* registering custom handler */
void myalarm(int signum) { ... } /* the handler */
alarm(30);                   /* sending alarm after 30 seconds */
```

man -s 7 signal (part)

Signal	Value	Action	Comment
SIGHUP	1	Term	Hangup detected on controlling terminal or death of controlling process
SIGINT	2	Term	Interrupt from keyboard
SIGQUIT	3	Core	Quit from keyboard
SIGILL	4	Core	Illegal Instruction
SIGABRT	6	Core	Abort signal from abort(3)
SIGFPE	8	Core	Floating point exception
SIGKILL	9	Term	Kill signal
SIGSEGV	11	Core	Invalid memory reference
SIGPIPE	13	Term	Broken pipe: write to pipe with no readers
SIGALRM	14	Term	Timer signal from alarm(2)
SIGTERM	15	Term	Termination signal
SIGUSR1	30,10,16	Term	User-defined signal 1
SIGUSR2	31,12,17	Term	User-defined signal 2
SIGCHLD	20,17,18	Ign	Child stopped or terminated
SIGCONT	19,18,25	Cont	Continue if stopped
SIGSTOP	17,19,23	Stop	Stop process
SIGTSTP	18,20,24	Stop	Stop typed at tty
SIGTTIN	21,21,26	Stop	tty input for background process
SIGTTOU	22,22,27	Stop	tty output for background process

UNIX pipelines: `pipe()`

- Goal data forwarding between processes (e.g. `ls -la | grep java`)
- Properties
 - Only in parent-child relations
 - Data stream: no message delimiters, no data types
 - Single direction stream (writer $\rightarrow$ reader), multiple readers are possible
 - Limited capacity: e.g.: 4 Kbyte (Linux < 2.6.11), 65 Kbyte (Linux >= 2.6.11)
- Implementation
 - A process creates a pipe (`pipe()`)
 - The kernel created the data structures and returns the read/write descriptors
 - The processes are communicate with the descriptors (`read()`, `write()`)
- Constrains, problems
 - No addressing, no type, no message delimiter
 - Only works with „relative” processes

UNIX named pipe, FIFO

- It solves the main problem of the simple pipes
 - Communication between independent tasks
 - Accessing to existing pipes are possible
- Properties
 - Not only available in parent-children relation
 - The operation is the same as the simple pipe
 - The creation is supported by the filesystem (mkfifo, mknod)
 - Bi-directional communication is possible (reading **and** writing)
- Example: communication with the `init` process
 - The named pipe in the filesystem:

```
prw----- 1 root root 0 Jan  1 12:38 /dev/initctl
```

- Further examples: [Introduction to Named Pipes](#)

UNIX System V IPC

- Goal: „standard” communication between tasks
 - Data exchange
 - Synchronization
- Definitions
 - Resource: the tool for the communication (see below)
 - Key: identifier to accessing the resource (an integer)
 - Common handler functions: `*ctl()`, `*get( ... key ...)`
 - Permission system
 - Creator, owner and their groups
 - The usual permission system: user, group, others
- Resources
 - Semaphores
 - Message queues more info: `man svipc ipc ipcs`
 - Shared memory

UNIX System V IPC: semaphores

- Goal: synchronization between processes
 - P() and V() operations
- Usage

```
sem_id = semget(key, number, options);
```

- It grants access to a given semaphore (with given key and options)
- The operations are described in the ops structure (details: man semop)

```
status = semop(sem_id, ops, ops_size);
```

- Multiple operations at the same time, even on multiple semaphores
- There are blocking and non-blocking P() methods
- It is possible to handle simple transactions also

UNIX System V IPC: message queues

- Goal: data exchange between processes

- Discrete messages with types
- No addressing, broadcast type

- Usage

```
msgq_id = msgget(key, options);
```

- It provides access to a message queue with a given key (and options)
- Sending a message (msg contains a type identifier also)

```
msgsnd(msgq_id, msg, size, options);
```

- Receiving a message

```
msgrcv(msgq_id, msg, size, type, options);
```

- With setting the type (integer) a simple filtering can be implemented
 - =0 the next message (any type)
 - >0 the next message with the given type
 - <0 the next message which type is less than equal

UNIX System V IPC: shared memory

- Goal: simple and fast data transfer between processes
 - Direct channel, without the kernel (shared memory range)
 - A dedicated region of the physical memory
 - Operation is based on PRAM model
- Usage

```
shm_id = shmget(key, size, options);
```

- Provides access to a given variable (identified by the key)
- Assignment to an own variable:

```
variable = (type) shmat(...);
```

- The variable is bond to the returned address
- Unbind: `shmdt ( address );`
- Mutual exclusion has to be applied (e.g.: with semaphores)

Network socket communication

- Goal: data transfer with addressing and standard protocols
 - Between any process – client-server architecture
 - Between local processes or between different machines
 - Multiple protocols and addressing modes
 - Originally it appeared in BSD UNIX (Berkeley sockets)
 - The Windows winsock and the POSIX socket is also based on this
- Definitions
 - Network socket: the endpoint of the communication
 - IP address and port number
- Usage

```
sfd = socket(domain, type, protocol);
server: bind(sfd, address, ...);
client: connect(sfd, address, ...);
server: listen(sfd, recommended_queue_size);
server: accept(sfd, cím, ...);
send(sfd, message, ...);
recv(sfd, message, ...);
shutdown(sfd);
```

The skeletons of the client and server programs

Client's code

```
s = socket()
```

```
connect(s)
```

```
send(s, msg)
```

```
recv(s, msg)
```

```
close(s)
```

Server's code

```
sfd1 = socket()
```

```
bind(sfd1)
```

```
listen(sfd1)
```

```
while
```

```
    sfd2 = accept(sfd1)
```

```
    fork()
```

```
    parent:    continue to loop
```

```
    children:  recv(sfd2)
```

```
                send(sfd2)
```

```
                close(sfd2)
```

```
                exit()
```

The basic forms of communication (summary)

- **PRAM model**
 - Shared memory
 - The operations are pipelined
- Synchronous transfer operations
- No addressing
- Applications:
 - Threads between a process
 - **Shared memory between processes**
- Pro-s
 - Fast, simple
 - No overhead after initialization
- Con-s
 - R-W and W-W conflicts
 - Synchronization between participants is necessary
 - Limited capacity (??)
- **Messaging systems**
 - It uses a communication system
 - Send and receive operations
 - Parallel operations are ordered
- Synchronous or asynchronous operation
- Addressing: direct, indirect, multi
- Applications
 - Mailbox
 - Pipeline
 - Message queue
 - **Network socket**
 - **Remote Procedure Call**
- Pro-s
 - Widely available
 - Between different machines
- Con-s
 - Communication errors has to managed
 - Slower